◆ HANDSOFFLABS · AUTODOCK & PACK · SYSTEMS NOTE

Why RK3588S, and how the whole loop stays instant

One walkthrough covering the chip choice, the recognition model plan (including telling Basmati from Sona Masoori), catalog onboarding, fleet rollout across branches, cloud sync, daily-usage analytics, and how discounts and free items ride the same pipe.

---

01 Why RK3588S, in plain terms

Think of it like choosing an engine for a delivery scooter, not a race car. A Jetson Orin Nano is the race-car engine — about 11x more raw AI power. But it needs a fan, it drinks 15–25W, and a single board costs more than the entire compute budget in your blueprint. RK3588S is the scooter engine: 5–8W, no fan needed, fits inside a sealed IP52 steel box, and costs roughly a third of the Jetson system. For a device that has to sit quietly inside a food-court kiosk all day on a fixed BOM, "enough power, cheap, cool-running" beats "maximum power" every time.

What RK3588S is actually doing inside the kiosk

JOB ON THE KIOSK	WHY RK3588S COVERS IT
Reading the two cameras + running the fast/deep recognition models	6 TOPS NPU
Driving the 10.1"/13.3" touch panel (checkout UI)	eDP output, no extra controller

Holding the local SKU catalog (~1,500 items) for offline matching	`up to 32GB RAM headroom`
Talking to the weight ADC, payment terminal, cloud sync	`RS-485 / networking on-chip`
Staying cool in a sealed steel enclosure, all day	`5–8W, passive cooling`

NOT THIS CHIP'S JOB

Driving the pack-and-seal motors for the 20kg drop. That's real-time, safety-critical actuator control — it belongs on its own small controller (an ESP32, same family you already use on Smart Soak) so a burst of AI inference never makes the packing arm stutter, and vice versa.

---

02 The model plan: cheap first, smart only when needed

The trick to "no latency" isn't one fast model — it's *not running the expensive model unless you have to*. Every item goes through the same funnel:

1. **Weight gate**
   The 4 load cells stay idle until something changes by more than 15g. No change, no camera wake-up, no compute spent. This alone kills most of the "always-on AI" cost.

2. **Weight pre-filter**
   The measured weight (±tolerance) narrows ~1,500 catalog SKUs down to a short candidate list before vision even runs — usually a handful of items, not the whole store.

3. **Fast path — MobileNetV4 + FAISS**
   A lightweight camera model turns the item into a numeric "fingerprint" (an embedding), compared against the candidate list's fingerprints already stored on-device. Target: under 30ms, covers roughly 9 in 10 items — anything visually distinct with a normal barcode-able package.

4. **Deep path — quantized VLM (Qwen2-VL, INT4)**
   Only triggers when the fast path isn't confident: loose bags, unusual angles, or lookalike items. Budget ~250ms. This is the path that needs the most testing on real RK3588 hardware, not the Mac.

On vector search specifically: keep `FAISS` running in-process on the kiosk (near-zero overhead for ~1,500 items) and save `Qdrant` for the cloud side, where it manages the

master catalog across every branch. Running a full database server on the kiosk itself just competes with the AI models for the same 8GB of RAM.

---

03 Telling Basmati from Sona Masoori — the hard case

This is the real technical challenge, and it's worth being honest about: a generic "what object is this" model was never trained to separate rice varieties. It'll cheerfully call all four "rice" and stop there. Solving it takes two extra ingredients layered onto the pipeline above, not a bigger generic model.

Ingredient 1 — a fine-grained fingerprint, not a generic one

Instead of one all-purpose embedding model, fine-tune it specifically on grain shots — Basmati, Sona Masoori, plain rice, idli rice — from your own onboarding photos, at consistent lighting and angle. This pulls those four fingerprints apart in vector space on purpose, the way a jeweler's loupe is built to separate near-identical stones rather than to recognize "a stone."

Ingredient 2 — density as a second, nearly-free signal

Weight ÷ how much space the item takes up in the tray gives a bulk-density number. Basmati (long, slender grains) sits at a different density than idli rice (short, parboiled, denser). This costs almost no compute — it's arithmetic on numbers you already have — and it's often more decisive than the image alone.

HOW THEY COMBINE

texture fingerprint + density number + "only match against what this store actually stocks" (store context) → a confidence score. High confidence → ring it up. Low confidence, or two lookalikes still tied → that's exactly when the deep VLM path earns its 250ms, by reasoning over the image in words ("long slender grains, low density → Basmati") instead of just pattern-matching.

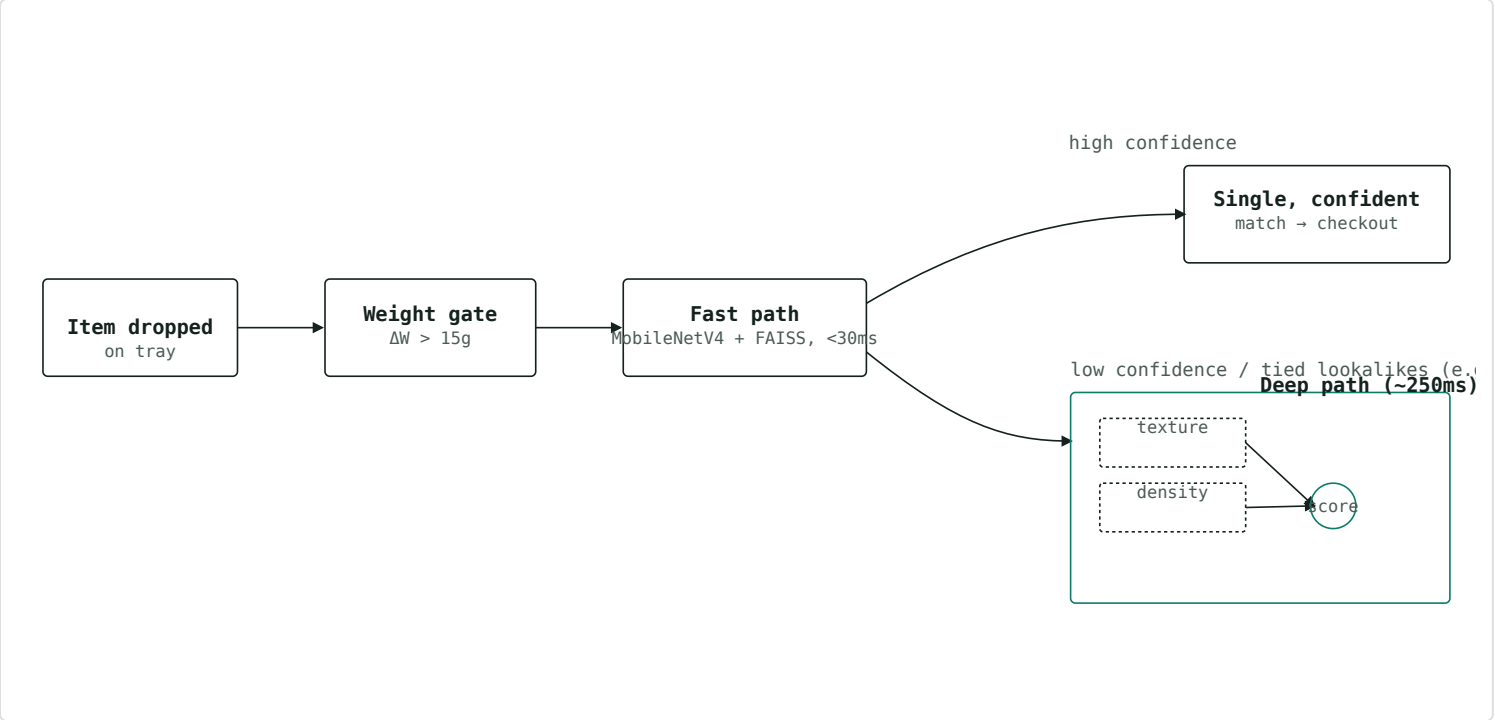

Fig 1 — weight gates the camera, weight pre-filters the candidates, and only genuinely ambiguous items (same-category lookalikes) pay the 250ms deep-path cost.

Practical next step: start collecting a small labeled photo set per grain variety now (multiple fill levels, angles, lighting) — that dataset is what the fine-tuned fingerprint model above is trained on. It matters more to accuracy than any chip decision.

---

04 Adding a new SKU — no retraining, ever

"No retraining" doesn't mean the system gets smarter on its own — it means adding a product never touches the underlying AI model at all. Only the reference list of fingerprints grows.

1. **Capture**
   4 photos of the new item + its weight, taken once, from a branch or HQ.
2. **Fingerprint**
   The same fixed embedding model turns those photos into vectors — nothing is trained, it's a lookup, so this takes seconds.
3. **Publish**
   The new vectors + price land in the cloud master catalog (Section 07) and get pushed to whichever kiosks stock that item.

4 **Live**

~30 seconds after publish, any kiosk that received it can recognize the item — same model, bigger reference list.

---

05 Making rollout feel instant

Two different things both need to feel instant, and they're solved differently:

Scanning at the counter — instant because it's local

The fast/deep path recognition (Section 02) never calls the cloud mid-transaction. Everything it needs is already sitting in the kiosk's local FAISS index. This is the actual mechanism behind "no latency during scanning" — it's not that the network is fast, it's that checkout doesn't use the network at all.

New kiosk or new SKU going live — instant because sync is small and incremental

- **First boot at a new site:** the kiosk authenticates with its device ID + branch ID, downloads that branch's active SKU subset (vectors are small — a few KB per item, not photos) and the current promotion rules, and is ready to trade in minutes, not days.
- **A single new SKU, on an already-running fleet:** pushed as a small delta over MQTT (the same channel your blueprint already planned for fleet sync) — every subscribed kiosk has it within roughly 30 seconds, no reboot, no reinstall.

---

06 Running many branches under one brand

Model this as three levels, and let each level own only what's genuinely local to it:

Who owns what

LEVEL	OWNS
Brand (HQ)	Master SKU catalog, master fingerprints, brand-wide pricing defaults

Branch	Which subset of the master catalog it actually stocks, branch-specific price overrides and promotions
Kiosk	A local copy of just its branch's active subset, so on-device search stays small and fast

A branch manager adding a promotion or a local price change never touches another branch, and never needs an engineer — it's a config change at the branch level that syncs down the same delta channel as new SKUs.

07 How catalog data actually reaches the cloud

Given your background with outbox patterns and event-driven sync, this will look familiar: each kiosk keeps a small local queue of things that happened (sales, new-item photos if captured on-device) and drains it whenever it has connectivity — never blocking a live transaction on a network call.

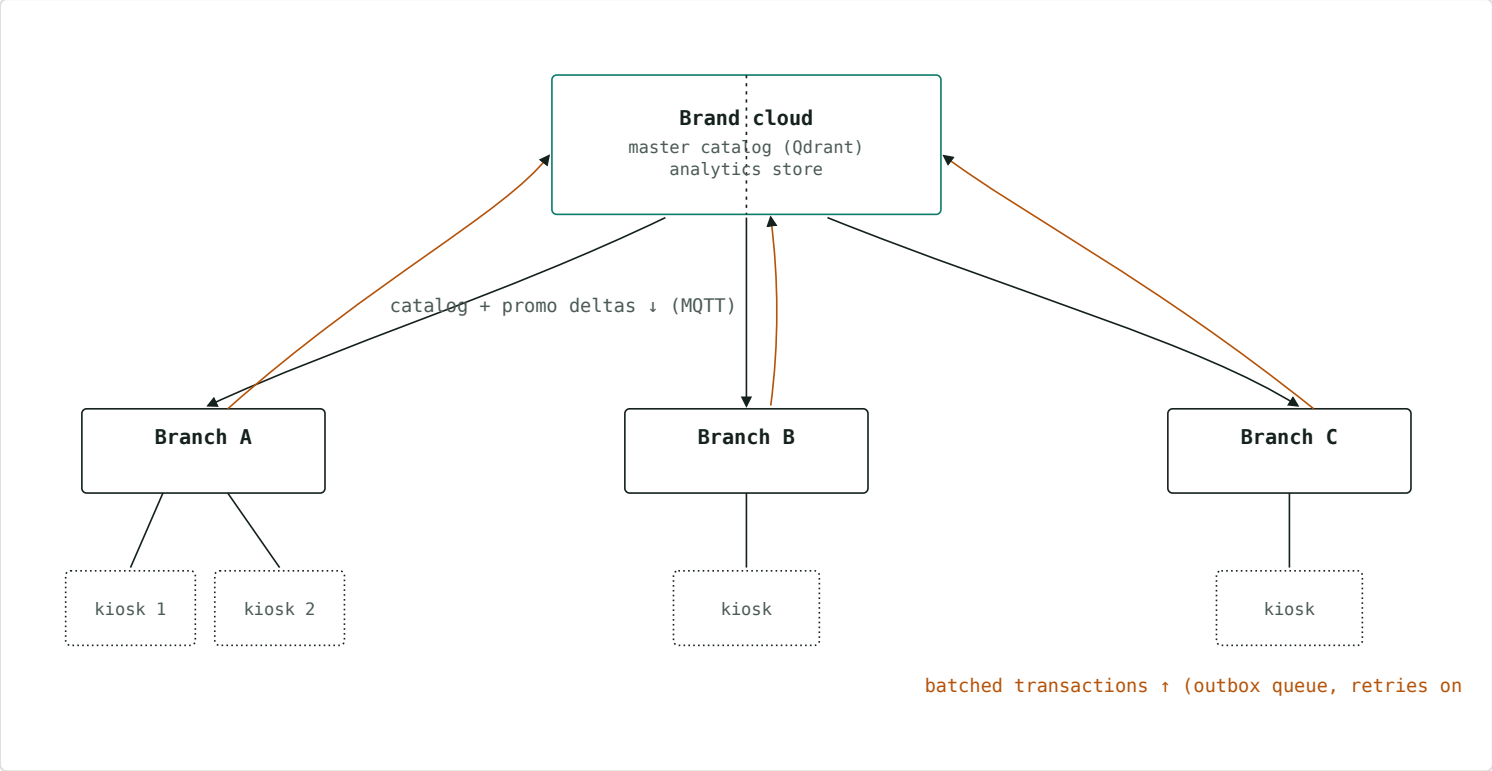

Fig 2 — catalog and promotion changes flow one way down to kiosks; sales and usage data flow the other way up, both asynchronously so a dropped connection never stalls a checkout.

08 Counting daily customers per device

Every completed checkout is one event: device ID, branch ID, items, amount, timestamp. It's written to the kiosk's local outbox the instant the transaction closes — before any network call — so counting never depends on connectivity being up at that moment.

1. **Local write**
   Transaction closes → one row in the kiosk's local queue. This is your source of truth even if the branch is offline for a day.
2. **Batched upload**
   The kiosk drains its queue to the cloud analytics store every few minutes (or immediately, if connectivity just came back).
3. **Roll-up**
   "Checkouts today, this device" is just a count filtered by device ID and date — the same shape gives you per-branch and per-brand totals for free, no separate pipeline.

---

09 Discounts and free items

These ride the exact same delta-sync channel as new SKUs (Section 05/07) — a promotion is just another small rule pushed down to the kiosks that should honor it, and evaluated locally so it still works offline.

Promotion types and how they resolve at checkout

TYPE	RESOLVED LOCALLY AS
% or ₹ off an item or category	a price-adjustment rule keyed to a SKU or SKU group, checked after identification, before payment
Buy-X-get-Y	a rule that watches the running cart, not a single item
Free sample / loyalty reward	a ₹0 price override on a flagged SKU, unlocked by scanning a phone number or loyalty code

WHY LOCALLY

If a branch loses connectivity mid-day, a promotion that only lived in the cloud would silently vanish. Evaluating it on-device — using the last-synced rule set — means the discount is exactly as reliable as the scanning itself.